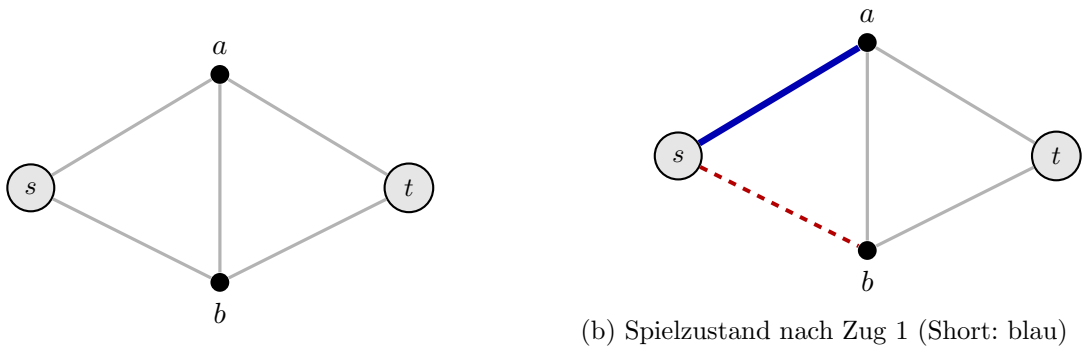
Technische Universität Berlin **SoSe 2026**
Fakultät II, Institut für Mathematik
Prof. Dr. Max Klimm
Dr. Sven Jäger, Dr. Alexander Lindermayr, Marcel Wack

Programmierprojekt Computerorientierte Mathematik II

Abgabe: Durch Vorstellung bis zum 04.07.2026

> **Hinweis:** Das Programmierprojekt wird in Dreiergruppen bearbeitet und bei einem Tutor oder einer Tutorin vorgestellt. Termine für die Vorstellung werden über ISIS bekannt gegeben.



(a) Ein Spielgraph (alle Kanten neutral)

(b) Spielzustand nach Zug 1 (Short: blau) und Zug 2 (Cut: rot gestrichelt)

Abbildung 1: Das Shannon-Switching-Spiel auf einem einfachen Graphen.

1. Das Shannon-Switching-Spiel

> **Hinweis:** Bitte lesen Sie die .pdf-Datei komplett und gründlich durch, bevor Sie mit der Bearbeitung beginnen.

Das Shannon-Switching-Spiel ist ein klassisches Zwei-Personen-Spiel auf Graphen, das von Claude Shannon in den 1950er Jahren eingeführt wurde. Ein Spieler versucht, eine Verbindung zwischen zwei Knoten aufzubauen, während der andere dies zu verhindern sucht.
Dieses Projekt gliedert sich in vier Teile: In **Teil 1** implementieren Sie die grundlegenden Datenstrukturen und Algorithmen. In **Teil 2** erstellen Sie eine spielbare Visualisierung. In **Teil 3** implementieren Sie eine *optimale* Computerstrategie für die klassische ungewichtete Variante. **Teil 4** ist der kreative Teil des Projekts: Sie entwickeln Heuristiken für das gewichtete Spiel, bei dem keine optimale Strategie bekannt ist. Die besten Strategien aller Gruppen werden in einer laufenden Bestenliste verglichen.

1.1. Spielregeln

Gegeben ist ein ungerichteter Graph $G = (V, E)$ mit zwei ausgezeichneten Knoten s (Quelle, engl. *source*) und t (Ziel, engl. *target*). Kanten können sich in einem von drei Zuständen befinden: *neutral*, *beansprucht* (von Short) oder *entfernt* (von Cut). Zu Beginn sind alle Kanten neutral. Zwei Spieler agieren abwechselnd, wobei **Short** beginnt:

- **Short** wählt eine neutrale Kante und *beansprucht* sie dauerhaft.
- **Cut** wählt eine neutrale Kante und *entfernt* sie dauerhaft aus dem Graphen.

Short gewinnt, sobald die von ihm beanspruchten Kanten einen Weg von s nach t bilden. **Cut gewinnt**, sobald im verbleibenden Graphen (neutrale und beanspruchte Kanten zusammen) kein s-t-Weg mehr existiert.

> **Hinweis:** Das Spiel wird auf ungerichteten, einfachen Graphen gespielt. Short macht den ersten Zug. Es ist möglich, dass das Spiel endet, bevor alle Kanten beansprucht oder entfernt wurden.

> **Hinweis:** Alle in diesem Projekt verwendeten Graphen sind zusammenhängend. Sie müssen daher keine Zusammenhangskomponenten des Ausgangsgraphen gesondert behandeln.

Da Short stets den ersten Zug macht, lässt sich jedes Spiel einer von zwei Klassen zuordnen:

- **Short-Spiel**: Short gewinnt – entweder durch seinen Startvorteil als erster Spieler oder weil Short sogar als zweiter Spieler gewinnen würde.
- **Cut-Spiel**: Cut hat eine Gewinnstrategie als zweiter Spieler.

Abbildung 1 zeigt ein Beispiel: Links sieht man den Spielgraph zu Beginn (alle Kanten neutral). Rechts ist ein Spielzustand nach dem ersten Zug jedes Spielers zu sehen: Short hat $\{s, a\}$ beansprucht, Cut hat $\{s, b\}$ entfernt. Short kann das Spiel noch gewinnen, da noch Wege über a nach t existieren.

1.2. Gewichtetes Spiel

Im gewichteten Shannon-Switching-Spiel trägt jede Kante e ein positives Gewicht $w(e) > 0$. Die Spielregeln bleiben identisch, jedoch zählt nun auch *wie* Short gewinnt: Short möchte das Gesamtgewicht des von ihm beanspruchten s-t-Weges *minimieren*, Cut möchte es *maximieren* (Short also zwingen, einen möglichst teuren Weg zu nehmen).
Im Gegensatz zur ungewichteten Variante ist für das gewichtete Spiel keine polynomielle optimale Strategie bekannt, es ist offen, ob eine solche überhaupt existiert. Hier sind Sie gefragt: Entwickeln Sie Heuristiken, die möglichst gute Ergebnisse erzielen. Die Datenstrukturen aus Teil 1 unterstützen das gewichtete Spiel von Anfang an über das `weight`-Feld der Kante.

Hinweis: Das klassische ungewichtete Shannon-Switching-Spiel ist ein Sonderfall des gewichteten Spiels, bei dem alle Kantengewichte null sind (im Code: `weight = 0.0`). In diesem Fall hängt der Gewinner einzig davon ab, ob Short überhaupt einen Weg aufbauen kann.

2. Schnittstelle

Ihre Aufgabe ist es, das Shannon-Switching-Spiel in Julia zu implementieren. Strukturieren Sie Ihr Programm dabei als **Julia-Paket**: Erstellen Sie eine `Project.toml` und legen Sie Ihren Quellcode im `src/`-Verzeichnis ab. Ein neues Julia-Paket lässt sich mit `Pkg.generate("PackageName")` in der Julia-REPL anlegen; dies erstellt `Project.toml` und das `src/`-Verzeichnis automatisch. Hierfür benötigen Sie:

1. Datenstrukturen zur Repräsentation des Spielgraphen und des Spielzustands,
2. die Spiellogik (Zugausführung, Gewinnbedingung).

2.1. Datenstrukturen

Die folgenden Datenstrukturen bilden die Grundlage Ihres Programms. Sie müssen mindestens diese Strukturen implementieren, können aber gerne weitere hinzufügen, wenn Sie dies für sinnvoll erachten. Auch weitere Felder in den Strukturen sind erlaubt, solange die geforderten Felder vorhanden sind und korrekt funktionieren.

Vertex

Ein `struct` mit dem Feld

`id::Int`: Eindeutige Kennung des Knotens.

Edge

Ein `mutable struct` mit den Feldern

`id::Int`: Eindeutige Kennung der Kante.

`u::Vertex`, `v::Vertex`: Die beiden Endpunkte der ungerichteten Kante.

`weight::Float64`: Kantengewicht (0 für das ungewichtete Spiel).

`state::Symbol`: Zustand der Kante: `:neutral`, `:short` (beansprucht) oder `:cut` (entfernt).

GameGraph

Ein `struct` mit den Feldern

`vertices::Vector{Vertex}`: Alle Knoten des Graphen.

`edges::Vector{Edge}`: Alle Kanten des Graphen.

`s::Vertex`: Quellknoten.

`t::Vertex`: Zielknoten.

GameState

Ein `mutable struct` mit den Feldern

`graph::GameGraph`: Der Spielgraph (mit veränderbaren Kantenzuständen).

`current_player::Symbol`: Der aktuelle Spieler: `:short` oder `:cut`.

`history::Vector{Tuple{Symbol, Edge}}`: Liste aller bisherigen Züge.

`winner::Union{Symbol, Nothing}`: Gewinner (`:short`, `:cut`) oder `nothing`, falls das Spiel noch läuft.

Mögliche Typstruktur

```
struct Vertex
    id::Int
end

mutable struct Edge
    id::Int
    u::Vertex
    v::Vertex
    weight::Float64
    state::Symbol   # :neutral, :short, :cut
end

struct GameGraph
    vertices::Vector{Vertex}
    edges::Vector{Edge}
    s::Vertex
    t::Vertex
end

mutable struct GameState
    graph::GameGraph
    current_player::Symbol
    history::Vector{Tuple{Symbol, Edge}}
    winner::Union{Symbol, Nothing}
end
```

2.2. Funktionen

Sie implementieren die folgenden Funktionen. Alle exportierten Funktionen müssen ausreichend dokumentiert sein.

`new_game(g::GameGraph)::GameState`: Erstellt einen neuen Spielzustand für den Graphen `g`. Alle Kanten sind neutral, Short beginnt, kein Gewinner.

`valid_moves(state::GameState)::Vector{Edge}`: Gibt alle neutralen Kanten zurück, die der aktuelle Spieler wählen darf.

`make_move!(state::GameState, e::Edge)::Nothing`: Führt den Zug des aktuellen Spielers auf Kante `e` aus: Short setzt `e.state = :short`, Cut setzt `e.state = :cut`. Aktualisiert `history`, wechselt den aktiven Spieler und prüft die Gewinnbedingung.

`check_winner(state::GameState)::Union{Symbol, Nothing}`: Gibt `:short` zurück, falls Shorts beanspruchte Kanten einen s-t-Weg umfassen; gibt `:cut` zurück, falls im verbleibenden Graphen kein s-t-Weg mehr existiert; sonst `nothing`.

Es ist optional, aber empfehlenswert, weitere Funktionen zu implementieren, etwa zur Generierung von Testgraphen.

`random_graph(n::Int, m::Int; weighted=false)::GameGraph`: Erzeugt einen zufälligen zusammenhängenden Graphen mit n Knoten und m Kanten. Knoten 1 ist die Quelle s, Knoten n ist das Ziel t. Im gewichteten Fall werden Kantengewichte gleichmäßig aus $[1, 10]$ gezogen.

> **Hinweis:** Achten Sie darauf, dass `make_move!` nur auf neutralen Kanten aufgerufen wird. Alle Züge sollten über `valid_moves` validiert werden, bevor sie ausgeführt werden.

3. Visualisierung

Ihr Programm muss das Shannon-Switching-Spiel **spielbar** machen: Zwei menschliche Spieler sollen abwechselnd Züge eingeben und das Spiel zu Ende spielen können. Wie Sie dies umsetzen, bleibt Ihnen weitgehend überlassen.
Eine Möglichkeit wäre `Gtk4` in Kombination mit `GtkObservables`. Weitere Informationen dazu gibt es in der Übung am 4. Juni; Sie können aber auch andere Ansätze wählen.
Folgende Ideen könnten Ihre Visualisierung verbessern, sie sind aber nicht verpflichtend:

- Einfärbung der Kanten nach Zustand (z. B. neutral grau, Short blau, entfernt rot oder unsichtbar).
- Anzeige des aktuellen Spielers, des Spielstatus und des Gewinners.
- Klick auf eine Kante zum Ausführen eines Zuges; Hervorhebung gültiger Züge.
- Schaltfläche zum Starten eines neuen Spiels.
- Laden vordefinierter Graphen.
- Grapheditor: Knoten und Kanten hinzufügen, verschieben, Gewichte setzen, s und t wählen.
- Computerstrategien aus Abschnitt 4.1 als spielbaren Gegner einbinden.

4. Klassisches Spiel – Optimale Strategie

Im klassischen ungewichteten Spiel kann eine optimale Computerstrategie effizient in polynomieller Zeit berechnet werden. In diesem Teil implementieren Sie diese.

4.1. Optimale Strategien für das klassische Spiel

Implementieren Sie optimale Computerstrategien für beide Spieler im *ungewichteten* Spiel.

Signaturen

```
short_strategy(state::GameState)::Edge
cut_strategy(state::GameState)::Edge
```

Sei G' der Graph, der alle neutralen und Short-beanspruchten Kanten aus dem aktuellen Spielzustand enthält.

Shorts optimale Strategie

Short hält zwei Bäume A_t und B_t mit derselben Knotenmenge $V \supseteq \{s, t\}$ aufrecht (Steiner-Bäume für die Terminals s und t), deren neutrale Kanten disjunkt sind. Solange Short diese Invariante aufrechterhalten kann, hat er eine Gewinnstrategie. Es genügt dabei, einen Teilgraphen $H \subseteq G'$ zu finden, der s und t enthält und auf dem zwei solche Bäume mit disjunkten neutralen Kanten existieren. Die Strategie beruht darauf, dass Short auf einen Cuts Züge reagieren kann, indem er diese Bäume repariert, um die Invariante zu erhalten. Da Short zuerst zieht und noch kein Cut-Zug vorliegt, simuliert Short beim ersten Zug einen virtuellen Cut-Zug auf der imaginären Kante $e^* = (s, t)$ und reagiert darauf mit der Reparaturstrategie. Das liefert Shorts echten ersten Zug.
Konkret: Sei a die Kante, die Cut soeben entfernt hat; beim ersten Zug gilt $a = e^* = (s, t)$.

- Falls $a \in A_t$: $A_t - \{a\}$ zerfällt in zwei Teilbäume $C_s \ni s$ und $C_t \ni t$. Da B_t denselben Graphen aufspannt, gibt es eine Kante $b \in B_t$ mit einem Endpunkt in C_s und einem in C_t. Short beansprucht b; der neue Hauptbaum ist $A'_t = A_t - \{a\} \cup \{b\}$.
- Falls $a \in B_t$: symmetrisch – Short repariert B_t mit einer Kante aus A_t.
- Falls $a \notin A_t \cup B_t$: Short beansprucht eine beliebige neutrale Kante auf einem s-t-Weg in G'.

Natürlich beruht hier alles darauf, dass Short die beiden Spannbäume finden kann. Wenn dies nicht möglich ist, hat Short nur dann noch eine Möglichkeit zu gewinnen wenn auch Cut keine Gewinnstrategie mehr hat.

Algorithm 1 Shorts optimale Strategie

```
1:  procedure short_strategy(state)
2:      G' ← neutralen und Short-beanspruchten Kanten aus state
3:      Berechne A_t, B_t: zwei Spannbäume von G' mit disjunkten neutralen Kanten
4:      a ← zuletzt von Cut entfernte Kante aus state.history; beim ersten Zug a = e* = (s, t)
5:      if a ∈ A_t then
6:          C_s, C_t ← zwei Teilbäume von A_t − {a}
7:          b ← eine Kante in B_t mit Endpunkten in C_s und C_t
8:      else if a ∈ B_t then
9:          C_s, C_t ← zwei Teilbäume von B_t − {a}
10:         b ← eine Kante in A_t mit Endpunkten in C_s und C_t
11:     else
12:         b ← eine neutrale Kante auf einem s-t-Weg in G'
13:     return b
```

Cuts optimale Strategie

Cut spielt eine ähnliche Strategie. Auch dieser Spieler reagiert auf Züge von Short mit einer Reparaturstrategie.
Die Idee: Cut verwaltet zwei disjunkte Kantenmengen A_t und B_t mit folgenden Eigenschaften:

1. Jeder s-t-Weg in G' enthält mindestens eine Kante aus A_t und mindestens eine aus B_t.
2. Jeder Kreis in G', der eine A_t-Kante enthält, enthält auch eine B_t-Kante (und umgekehrt).

Solange diese Invariante gilt, können Shorts beanspruchte Kanten allein keinen s-t-Weg bilden: jeder vollständige Weg braucht noch je eine neutrale A_t- und B_t-Kante, die Cut noch entfernen kann.
Wichtig: A_t und B_t sind *keine Spannbäume* von G'. Für Spannbäume ist Eigenschaft 1 im Allgemeinen *nicht* erfüllt – ein Spannbaum enthält einen eindeutigen s-t-Weg, aber s-t-Wege, die ausschließlich Nicht-Baumkanten benutzen, wären nicht abgedeckt. Wie A_t und B_t bestimmt werden, wird weiter unten erläutert.
Wenn Short eine Kante $a \in A_t$ beansprucht, verliert höchstens ein s-t-Weg P seine letzte neutrale A_t-Kante (nämlich a). Da B_t ebenfalls jeden s-t-Weg schneidet, enthält P noch eine neutrale Kante $b \in B_t$. Cut entfernt b. Die aktualisierte Menge $A'_t = A_t \setminus \{a\} \cup \{b\}$ stellt die Invariante wieder her: der Weg P enthält jetzt $b \in A'_t$, und alle anderen Wege haben weiterhin ihre A_t-Kanten. Da b bereits von Cut entfernt wurde, kann Short es nicht beanspruchen – $b \in A'_t \cap B_t$ ist dauerhaft blockiert.

Berechnung von A_t und B_t. Die Berechnung von A_t und B_t ist das eigentliche algorithmische Kernproblem. Sei T ein Spannbaum von G'. Für jede Baumkante $f \in T$ heißt $D(f, T)$ der *Fundamentalschnitt* von f: die Menge aller Kanten in G', die den von $T \setminus \{f\}$ induzierten Schnitt kreuzen. Die Fundamentalschnitte $\{D(f, T) \mid f \in T\}$ spielen für die Schnittstruktur von G' dieselbe Rolle wie die Fundamentalkreise für die Kreisstruktur. Die Mengen A_t und B_t können mit einem Verfahren berechnet werden, das dem Algorithmus aus Anhang A entspricht, jedoch auf der Schnittstruktur operiert: Statt Kreisen und Spannbäumen arbeitet man mit Schnitten und *Ko-Spannbäumen* (Komplementen von Spannbäumen), und die Augmentierungsschritte verwenden Fundamentalschnitte statt Fundamentalkreise.

Algorithm 2 Cuts optimale Strategie

```
1:  procedure cut_strategy(state)
2:      G' ← neutrale und Short-beanspruchte Kanten aus state
3:      Berechne A_t, B_t: zwei disjunkte s-t-Trennmengen in G' mit Eigenschaft 2 (siehe Hinweis)
4:      a ← zuletzt von Short beanspruchte Kante                    (aus state.history)
5:      if a ∈ A_t then
6:          P ← ein s-t-Weg in G', der a enthält, aber keine weitere neutrale A_t-Kante
7:          if P existiert then
8:              b ← eine neutrale Kante aus B_t ∩ P
9:          else
10:             b ← eine beliebige neutrale Kante aus B_t
11:     else if a ∈ B_t then
12:         P ← ein s-t-Weg in G', der a enthält, aber keine weitere neutrale B_t-Kante
13:         if P existiert then
14:             b ← eine neutrale Kante aus A_t ∩ P
15:         else
16:             b ← eine beliebige neutrale Kante aus A_t
17:     else
18:         b ← eine beliebige neutrale Kante aus A_t ∪ B_t
19:     return b
```

> **Hinweis:** Die beschriebenen Strukturen müssen nicht immer existieren – etwa wenn G' keinen s-t-Weg mehr enthält oder der Graph zu wenig Kanten hat. In diesem Fall hat der jeweilige Spieler keine Gewinnstrategie; `short_strategy` und `cut_strategy` dürfen dann einen beliebigen gültigen Zug zurückgeben.

5. Gewichtetes Spiel – Strategiewettbewerb

Das gewichtete Shannon-Switching-Spiel ist der kreative Teil dieses Projekts. Da keine optimale polynomielle Strategie bekannt ist, sind Sie gefragt: Entwickeln Sie Heuristiken, die möglichst gute Ergebnisse erzielen. Die Strategien aller Gruppen werden in einer **laufenden Bestenliste** gegeneinander antreten.

5.1. Bewertung

Im gewichteten Spiel zählt nicht nur, *ob* Short einen s-t-Weg baut, sondern auch dessen Gesamtgewicht. Konkret wird jede Strategie auf zufälligen zusammenhängenden gewichteten Graphen gegen andere Gruppen antreten:

> **Hinweis:** Im Wettbewerb wird das Spiel immer bis zur letzten Kante gespielt, es gibt keine vorzeitige Beendigung. Beide Strategiefunktionen werden also so lange aufgerufen, bis keine neutrale Kante mehr übrig ist. Erst danach wird ausgewertet, ob Shorts beanspruchte Kanten einen s-t-Weg bilden, und dessen Gewicht berechnet.

- Short's **Punktzahl** in einer Partie ist das Gesamtgewicht des *kürzesten* s-t-Weges unter Shorts beanspruchten Kanten am Spielende. Gelingt es Short nicht, s und t zu verbinden, erhält er eine hohe Strafpunktzahl.
- Jedes Duell zweier Gruppen wird **zweimal** auf demselben Graphen gespielt: einmal übernimmt Gruppe A die Rolle von Short und Gruppe B die Rolle von Cut, einmal umgekehrt. So tritt jede Gruppe in jedem Duell genau einmal als Short und einmal als Cut an.
- **Gewinner** eines Duells ist die Gruppe mit der *niedrigeren* Short-Punktzahl. Die Bestenliste wertet die Anzahl gewonnener Duelle; bei Gleichstand entscheidet die durchschnittliche Short-Punktzahl (*niedriger ist besser*).

5.2. Schnittstelle für den Wettbewerb

Ihre Wettbewerbsstrategie muss die folgende Schnittstelle implementieren:

Signaturen für den Wettbewerb

```
const TEAM_NAME::String = "???"   # Tragen Sie hier Ihren Teamnamen ein

weighted_short(state::GameState)::Edge
weighted_cut(state::GameState)::Edge
```

Sie können Ihre Abgabe durch erneutes Absenden jederzeit aktualisieren.

> **Hinweis:** Programme, die zu viel Arbeitsspeicher verbrauchen oder für einen einzelnen Zug länger als ca. **2 Sekunden** benötigen, werden automatisch beendet und verlieren das laufende Spiel. Achten Sie daher auf eine effiziente Implementierung Ihrer Strategie.

5.3. Abgabe und Bestenliste

Reichen Sie Ihre Wettbewerbsstrategie mit dem folgenden Befehl ein:

```
comajudge submit -t your-file.jl -p Shannon
```

Der Wettbewerb findet **täglich um 18 Uhr** statt. Das aktuelle Ergebnis der Bestenliste können Sie danach mit

```
comajudge result -p Shannon
```

abrufen.

> **Hinweis:** Ihr Programm muss nicht die Spiellogik oder Visualisierung enthalten, sondern nur die Funktionen `weighted_short` und `weighted_cut` für die Wettbewerbsstrategie. Hilfsfunktionen und -strukturen, die Sie für die Entwicklung Ihrer Strategie benötigen, können Sie gerne in derselben Datei implementieren.

6. Zusammenfassung

Zusammenfassend brauchen Sie für eine erfolgreiche Abgabe:

- Die Datenstrukturen `Vertex`, `Edge`, `GameGraph` und `GameState`.
- Die Funktionen `new_game`, `valid_moves`, `make_move!` und `check_winner`.
- Eine spielbare Visualisierung (zwei Spieler können eine vollständige Partie spielen).
- Optimale Computerstrategien `short_strategy` und `cut_strategy` für das klassische Spiel.
- Wettbewerbsstrategien `weighted_short` und `weighted_cut` sowie die Konstante `TEAM_NAME` für den Wettbewerb. Für die Abgabe muss die Strategie erfolgreich über den comajudge abgegeben worden sein.
- Dokumentierte exportierte Funktionen sowie Tests zur Korrektheit.
- Das Programm als Julia-Project strukturiert (`Project.toml`, `src/`-Verzeichnis).

A. Berechnung zweier Spannbäume mit disjunkten neutralen Kanten

Das Berechnen von zwei Spannbäumen sollte jedem bekannt sein, schwieriger ist es, zwei Spannbäume mit disjunkten neutralen Kanten zu finden. Hierfür modifiziert man beide Spannbäume schrittweise, bis diese *maximal distant* sind. Was im besten Fall bedeutet, dass diese disjunkt sind. Hier stellen wir einen Algorithmus von Kishi und Kajitani vor, der genau maximal distante Spannbäume berechnet.

Zwei Spannbäume T_1 und T_2 eines Graphen heißen *maximal distant*, falls kein Spannbaumpaar einen größeren Abstand $d(T_1, T_2) := |T_1 \setminus T_2|$ besitzt. Da jeder Spannbaum desselben Graphen gleich viele Kanten enthält, gilt stets $|T_1 \setminus T_2| = |T_2 \setminus T_1|$, der Abstand ist also wohldefiniert. Besitzt G' zwei kantendisjunkte Spannbäume, so sind die maximal distanten Spannbäume kantendisjunkt.

Für den Algorithmus benötigen wir folgende Begriffe. Eine *Sehne* von T ist eine Kante in $E(G') \setminus T$. Eine Kante, die Sehne beider Bäume T_1 und T_2 ist (also in keinem der beiden liegt), heißt *gemeinsame Sehne*. Der *Fundamentalkreis* $\mathrm{FC}(e, T)$ einer Sehne e bezüglich T ist die Menge der T-Kanten auf dem eindeutigen Kreis in $T \cup \{e\}$. Für eine gemeinsame Sehne e definieren wir Schichten rekursiv:

$$L_e^1 \;=\; \mathrm{FC}(e, T_1), \qquad L_e^{k+1} \;=\; \bigcup_{f \,\in\, L_e^k} \mathrm{FC}(f,\, T_{\mathrm{alt}}),$$

wobei $T_{\mathrm{alt}} = T_2$ für ungerades k und $T_{\mathrm{alt}} = T_1$ für gerades k.

Der Algorithmus von Kishi und Kajitani verarbeitet gemeinsame Sehnen nacheinander und erhöht dabei $d(T_1, T_2)$ schrittweise um 1, bis keine gemeinsamen Sehnen mehr existieren. Das Kernstück ist AUGMENT (Algorithmus 4), das für eine gemeinsame Sehne e prüft, ob eine solche Verbesserung möglich ist, und sie gegebenenfalls durchführt.

Algorithm 3 Maximal distante Spannbäume (Kishi-Kajitani)

```
1: procedure MaximallyDistantTrees(G', T_1, T_2)
2:     changed ← true
3:     while changed do
4:         changed ← false
5:         for all gemeinsame Sehne e ∈ E(G') \ T_1 \ T_2 do
6:             if Augment(T_1, T_2, e) then
7:                 changed ← true
8:     return (T_1, T_2)
```

Die Korrektheit der Tauschregel beruht darauf, dass $chain = [c_1, \ldots, c_{k-1}, f]$ abwechselnd T_1- und T_2-Kanten enthält ($c_1 \in T_1 \setminus T_2$, $c_2 \in T_2 \setminus T_1$ usw., $f \in T_1 \cap T_2$), sodass jeder Tausch $d(T_1, T_2)$ um genau 1 erhöht. Gibt MAXIMALLYDISTANTTREES ein Paar ohne gemeinsame Sehnen zurück, sind T_1 und T_2 kantendisjunkt. Für Shorts Strategie genügt es zu prüfen, ob keine *neutrale* Kante in beiden Bäumen liegt; andernfalls befindet sich Short in einer verlorenen Position und darf einen beliebigen gültigen Zug wählen.

Algorithm 4 Augmentierung entlang einer gemeinsamen Sehne

1: **procedure** Augment(T_1, T_2, e)
2: $par \leftarrow \{\}$; $L \leftarrow \mathrm{FC}(e, T_1)$; $L_{\text{prev}} \leftarrow \emptyset$; $k \leftarrow 1$
3: **while** $L \neq L_{\text{prev}}$ **do**
4: $L_{\text{prev}} \leftarrow L$
5: $T_{\text{alt}} \leftarrow T_2$, falls k ungerade; sonst $T_{\text{alt}} \leftarrow T_1$
6: **if** $L \cap T_{\text{alt}} \neq \emptyset$ **then**
7: $f \leftarrow$ beliebige Kante aus $L \cap T_{\text{alt}}$
8: $x \leftarrow f$; $chain \leftarrow [f]$
9: **while** $x \in par$ **do**
10: $x \leftarrow par[x]$; $chain \leftarrow [x] \cdot chain$
▷ $chain = [c_1, \ldots, c_{k-1}, f]$ mit $c_1 \in L_e^1$
11: $T_1 \leftarrow T_1 \cup \{e\} \cup \{chain_i : i \text{ gerade}\} \setminus \{chain_i : i \text{ ungerade}\}$
12: $T_2 \leftarrow T_2 \cup \{chain_i : i \text{ ungerade}\} \setminus \{chain_i : i \text{ gerade}\}$
13: **return true**
14: **for** $g \in L$ **do**
15: **for** $f' \in \mathrm{FC}(g, T_{\text{alt}}) \setminus L$ **do**
16: $L \leftarrow L \cup \{f'\}$; $par[f'] \leftarrow g$
17: $k \leftarrow k + 1$
18: **return false**